COLLEGE OF APPLIED BUSINESS AND TECHNOLOGY

Kathmandu, Nepal

Laboratory Assignment Log Sheet of B.SC. CSIT 5th Semester

Subject: CSC 328: Simulation & Modeling

Lab No.	Title		
1.	Implement application of Markov Chain.		
2.	Implement application of Monte Carlo Method		
3.	Testing of Random Numbers		
	A.	Testing of Uniformity	
		i .	Implementation of Kolmogorov-Smirnov Test (K-S Test)
		ii.	Implementation of Chi Square Test.
	B.	Testing of Independence	
		i.	Implementation of Poker Test.
4.	Implement Linear Congruential Method for random number generation		
5.	Simulation of single server queue system using GPSS		
	i.	Barber Shop simulation to simulate one day of operation of a barber	
	ii.	Barber Shop Simulation (simple)	
	iii.	Mechanic Shop simulation	
	iv.	Telephone System Simulation	
	v.	Turnstile of Football Stadium	
	vi.	Manufacturing Shop	

LAB 1: IMPLEMENTATION OF THE APPLICATION OF MARKOV CHAIN

AIM: To implement the Markov Chain using C language

DESCRIPTION

• A Markov chain is a discrete-time process for which the future behavior, given the past and the present, only depends on the present and not on the past.
• A Markov Chain is a random process with the property that the next state depends only on the current state.
• Markov chains are widely used in various fields, including physics, computer science, economics, etc.

ALGORITHM

• Step 1: Define the State Space
• Step 2: Create a Transition Matrix:
  - Ensure that the sum of probabilities in each row of the matrix is equal to 1, as the system must transition to some state in the next step.
• Step 3: Specify the Initial State Distribution $X^{[0]} = [\ i\ \ j]$
• Step 4: Compute the n-Step Transition Matrix (Optional)
  - $X^{[n]} = X^{[n-1]} P$
  - OR $X^{[n]} = X^{[0]} P^n$
• Step 5: Analyze the Markov Chain:

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   // Define the transition matrix
   double P[2][2];
   printf("Enter the transition matrix (e.g., 0.7 0.3 0.4 0.6 for a 2x2 matrix):\n");
   scanf("%lf %lf %lf %lf", &P[0][0], &P[0][1], &P[1][0], &P[1][1]);

   // Print the transition matrix
   printf("Transition matrix:\n");
   printf("%f %f\n", P[0][0], P[0][1]);
   printf("%f %f\n", P[1][0], P[1][1]);

   // Define the current state matrix
   float x, y;
   printf("Enter the current state matrix (e.g., 1 0 for state 1):\n");
```

```
    scanf("%f %f", &x, &y);

    // Generate the Markov chain
    int n;
    printf("Enter the number of steps to generate:\n");
    scanf("%d", &n);

    printf("State sequence probabilities:\n");
    printf("%f %f\n", x, y);

    for (int i = 1; i <= n; i++) {
        // Generate the next state
        // This calculates the matrix multiplication: [x, y] * P
        float new_x = P[0][0] * x + P[1][0] * y;
        float new_y = P[0][1] * x + P[1][1] * y;

        printf("%f %f\n", new_x, new_y);

        x = new_x;
        y = new_y;
    }

    return 0;
}
```

OUTPUT

```
yanil@inspiron-14:~/Desktop/Simulation_lab$ ./a.out
Enter the transition matrix (e.g., 0.7 0.3 0.4 0.6 for a 2x2 matrix):
0.1
0.9
0.4
0.6
Transition matrix:
0.100000 0.900000
0.400000 0.600000
Enter the current state matrix (e.g., 1 0 for state 1):
0
1
Enter the number of steps to generate:
3
State sequence probabilities:
0.000000 1.000000
0.400000 0.600000
0.280000 0.720000
0.316000 0.684000
```

LAB-2: IMPLEMENT MONTE-CARLO METHOD

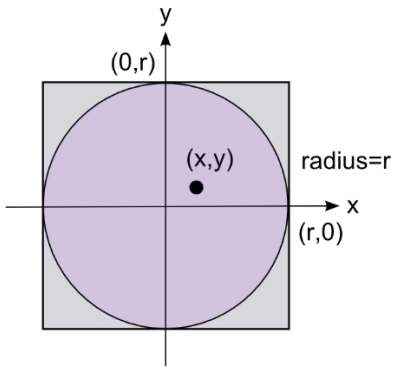
AIM:

To calculated the value of π based on the number of sample points specified by the user.

DESCRIPTION:

- Monte Carlo simulation is a computerized mathematical technique to generate random sample data based on some known distribution for numerical experiments.
- This method is applied to risk quantitative analysis & decision-making problems.
- This method is used by professionals of various profiles such as finance, project management, energy, manufacturing, engineering, research & development, transportation etc.

Determining the value of PI using Monte Carlo Method



• Area of quadrant of circle / Area of rectangle = No. of points inside circle/ No. of points inside the rectangle

or $1/4\ \pi r^2 / r^2 = n/N$

$\pi = 4n/N$

• We use random number generation to determine the sample points that lie inside the curve. Let (x 0, y0) be an initial guess for the sample point then from a linear congruential method of random number generation:

$X_{i+1} = (ax_i + c) \bmod m$ &

$Y_{i+1} = (ay_i + c) \bmod m$

• Where a & c are constants, m is the upper limit of generated random number.

• If $x^2+y^2 \leq 1$, then increment n. This checks if a randomly generated points (x, y) lies inside the circle of radius 1.

Algorithm for Estimating π Using Monte Carlo Method

1. Input:
    - o Number of sample points (num_points).

2. Initialize:
    - o Constants for the linear congruential generator (LCG):
        - ▪ A = 1664525
        - ▪ C = 1013904223
        - ▪ M = 4294967296 (which is 2^{32})
    - o Seed value for LCG (seed = 123456789).
    - o Counter for points inside the quarter circle (count_inside_circle = 0).
3. Prompt User for Input:
    - o Ask the user to enter the number of sample points (num_points).
4. Generate Random Points and Check if Inside Quarter Circle:
    - o For each point from i = 0 to i < num_points:
        - ▪ Generate a random number for x:
            - ▪ Update seed using LCG formula: seed = (A * seed + C) % M
            - ▪ Normalize x to be between 0 and 1: x =(double)seed / (double)M
        - ▪ Generate a random number for y:
            - ▪ Update seed using LCG formula: seed = (A * seed + C) % M
            - ▪ Normalize y to be between 0 and 1: y = (double)seed / (double)M
        - ▪ Check if the point (x, y) is inside the quarter circle:
            - ▪ If x2+ y2 <= 1, increment count_inside_circle.
5. Estimate π:
    - o Calculate the estimate for π using the formula:
    - o {pi_estimate} = 4.0 {count_inside_circle}/{num_points}
6. Output:
    - o Print the estimated value of π

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
// Constants for the linear congruential generator
#define A 1664525
#define C 1013904223
#define M 4294967296 // 2^32
int main () {
   int num_points;
   // Prompt the user for the number of sample points
   printf("Enter the number of sample points: ");
   scanf("%d", &num_points);
   unsigned int seed = 123456789; // Initial seed value
   int count_inside_circle = 0;
   for (int i = 0; i < num_points; i++) {
   // Linear Congruential Generator
      seed = (A * seed + C) % M;
      double x = (double)seed / (double)M;
      // Generate the second random number
      seed = (A * seed + C) % M;
```

```
        double y = (double)seed / (double)M;
        if (x * x + y * y <= 1.0) {
            count_inside_circle++;
        }
    }
    double pi_estimate = 4.0 * (double)count_inside_circle / (double)num_points;
    printf("Estimated value of Pi: %f\n", pi_estimate);
    return 0;
}
```

OUTPUT

```
yanil@inspiron-14:~/Desktop/Simulation_lab$ gcc 2.c
yanil@inspiron-14:~/Desktop/Simulation_lab$ ./a.out
Enter the number of sample points: 99
Estimated value of Pi: 3.191919
```

LAB 3.A(i) IMPLEMENTATION OF KOLMOGOROV-SMIRNOV TEST (K-S TEST)

AIM:

Program to implement Kolmogorov-Smirnov Test (K-S Test).

ALGORITHM:

- Step 1: Rank the data from smallest to largest. Let Ri denote ith smallest observation, so that $R_1 <= R_2 <= R_3 <= ... <= R_N$
- Step 2: Compute D+ = max {i /N – Ri } and D- = max { Ri – (i-1)/N}
- Step 3: Find D = max {D+, D-}
- Step 4: Determine the critical value, Dα, from Table for the specified value of α and the sample size N.
- Step 5: If the sample statistic D is greater than the tabulated value of Dα ie. (D>Dα), the null hypothesis that the data are a sample from uniform distribution is rejected.
  If D <= Dα, conclude that no difference has been detected between the true distribution of
  { R1 R2, ,• • •, Rn } and the uniform distribution. Hence the null hypothesis is accepted. (i.e not rejected).

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Function to calculate D+
double dplus(double num[], double d1[], int length, double n) {
   for (int i = 0; i < length; i++) {
      d1[i] = ((i + 1) / n) - num[i];
   }
   double d1max = d1[0];
   for (int i = 0; i < length; i++) {
      if (d1max <= d1[i]) {
         d1max = d1[i];
      }
   }
   return d1max;
}

// Function to calculate D-
```

```
double dminus(double num[], double d2[], int length, double n) {
    for (int i = 0; i < length; i++) {
        d2[i] = num[i] - (i / n);
    }
    double d2max = d2[0];
    for (int i = 0; i < length; i++) {
        if (d2max <= d2[i]) {
            d2max = d2[i];
        }
    }
    return d2max;
}

// Kolmogorov-Smirnov Test function
void kst(double num[], int length, double d_critical) {
    // Sorting the array
    for (int i = 0; i < length; i++) {
        for (int j = i + 1; j < length; j++) {
            if (num[i] > num[j]) {
                double temp = num[i];
                num[i] = num[j];
                num[j] = temp;
            }
        }
    }
    double d1[length];
    double d2[length];
    double n = (double) length;
    double d1max = dplus(num, d1, length, n);
    double d2max = dminus(num, d2, length, n);
    double d = (d1max > d2max) ? d1max : d2max;
    printf("D = %f\n", d);
    // Hypothesis Testing
    printf("D_critical = %f\n", d_critical);
    if (d > d_critical) {
        printf("Null hypothesis rejected.\n");
    } else {
        printf("Null hypothesis accepted.\n");
    }
}
int main() {
    printf("Kolmogorov Test\n");
    int n;
    printf("Enter number of elements to compute for test: \t");
    scanf("%d", &n);
    double num[n];
    printf("Enter the elements:\n");
```

```
    for (int i = 0; i < n; i++) {
        scanf("%lf", &num[i]);
    }
    double d_critical;
    printf("Enter the tabulated value of D_alpha: ");
    scanf("%lf", &d_critical);
    kst(num, n, d_critical);
    return 0;
}
```

OUTPUT

```
yanil@inspiron-14:~/Desktop/Simulation_lab$ gcc 3a.c
yanil@inspiron-14:~/Desktop/Simulation_lab$ ./a.out
Kolmogorov Test
Enter number of elements to compute for test:   4
Enter the elements:
0.23
0.34
0.45
0.56
Enter the tabulated value of D_alpha: 0.999
D = 0.440000
D_critical = 0.999000
Null hypothesis accepted.
```

LAB 3.A(ii) IMPLEMENTATION OF CHI SQUARE TEST

AIM:

Program to implement Chi Square Test.

ALGORITHM:

Algorithm for Chi-Square Test

1. **Define the Hypotheses**:
   - **Null Hypothesis (H0):** The observed frequencies follow the expected distribution.
   - **Alternative Hypothesis (H1):** The observed frequencies do not follow the expected distribution.
2. **Collect Data:**
   - Gather the observed frequencies for each category.
   - Determine the expected frequencies for each category.
3. **Calculate the Chi-Square Statistic:**
   - For each category, calculate the difference between the observed and expected frequencies.
   - Square the difference.
   - Divide the squared difference by the expected frequency.
   - Sum these values for all categories to obtain the chi-square statistic ($\chi 2$).

The formula for the chi-square statistic is:
$\chi^2 = \sum (Oi - Ei)^2 / Ei$

Where:

- Oi is the observed frequency for category i.
- Ei is the expected frequency for category i.

4. **Determine the Degrees of Freedom:**
   - The degrees of freedom for a chi-square test is calculated as the number of categories minus 1.
   - Degree of freedom=n-1
5. **Set the Significance Level:**
   - Choose a significance level (α), typically 0.05.
6. **Find the Critical Value:**
   - Use a chi-square distribution table to find the critical value corresponding to the chosen significance level and degrees of freedom.
7. **Compare the Chi-Square Statistic to the Critical Value:**
   - If the chi-square statistic is greater than the critical value, reject the null hypothesis.

- If the chi-square statistic is less than or equal to the critical value, fail to reject the null hypothesis.ie. the data given are uniformly distributed.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define NUM_BINS 10 // number of bins for chi-square test
int main() {
   int num_random, i;
   double rand_num, expected_frequency, chi_square, chi_critical;
   int observed_frequency[NUM_BINS] = {0};
   // Ask user for total number of numbers
   printf("Enter total number of numbers: ");
   scanf("%d", &num_random);
   double num_sequence[num_random];
   // Ask user for the number sequence
   printf("Enter the numbers (between 0 and 1):\n");
   for (i = 0; i < num_random; i++) {
      scanf("%lf", &num_sequence[i]);
   }
   // Count the frequencies in bins
   for (i = 0; i < num_random; i++) {
      if (num_sequence[i] >= 0 && num_sequence[i] < 1) {
         observed_frequency[(int)(num_sequence[i] * NUM_BINS)]++;
      } else {
         printf("Number %f out of range [0, 1). Exiting.\n", num_sequence[i]);
         return 1;
      }
   }
   // Calculate expected frequency and chi-square value
   expected_frequency = (double) num_random / NUM_BINS;
   chi_square = 0.0;
   for (i = 0; i < NUM_BINS; i++) {
      chi_square += pow(observed_frequency[i] - expected_frequency, 2) /
expected_frequency;
   }
   // Ask user for the critical value of chi-square at the given significance level
   printf("\nEnter the critical value of chi-square: ");
   scanf("%lf", &chi_critical);
   // Print results
   printf("\n\nChi-square value: %f\n", chi_square);
   if (chi_square > chi_critical) {
      printf("The numbers do not pass the chi-square test at the given significance
level.\n");
      printf("Null hypothesis rejected: The distribution is not uniform.\n");
   } else {
```

```
        printf("The numbers pass the chi-square test at the given significance level.\n");
        printf("Null hypothesis accepted: The distribution is uniform.\n");
    }
    return 0;
}
```

OUTPUT

```
yanil@inspiron-14:~/Desktop/Simulation_lab$ ./a.out
Enter total number of numbers: 3
Enter the numbers (between 0 and 1):
0.34
0.78
0.23

Enter the critical value of chi-square: 7.7


Chi-square value: 7.000000
The numbers pass the chi-square test at the given significance level.
Null hypothesis accepted: The distribution is uniform.
```

LAB 3.B(i) IMPLEMENTATION OF POKER TEST

AIM

Program to implement Poker Test

ALGORITHM

1. **Start**
2. **Define Categories:**
   - Define the possible patterns for three-digit numbers:
     - Three different digits (e.g., 123).
     - Exactly one pair of like digits (e.g., 112).
     - Three like digits (e.g., 111).
3. **Calculate Probabilities:**
   - Calculate the theoretical probabilities for each category:
     P(3 different digits) = P(2nd diff. from 1st) * P(3rd diff. from 1st and 2nd) = (0.9) (0.8) = 0.72
     P(3 like digits) = P(2nd digit same as 1st) * P(3rd digit same as 1st)= (0.1) (0.1) = 0.01
     P(exactly one pair) = 1 - 0.72 - 0.01 = 0.27
4. **Generate Sample Data:**
   - Generate a sequence of three-digit numbers. This can be done through simulation or using provided data.
5. **Classify Numbers:**
   - For each number in the sequence, classify it into one of the three categories:
     o Check if all three digits are different.
     o Check if exactly two digits are the same.
     o Check if all three digits are the same.
6. **Count Observed Frequencies:**
   - Count the observed frequency for each category:
     o observed_three_diff for three different digits.
     o observed_one_pair for exactly one pair of like digits.
     o observed_three_like for three like digits.

7. **Calculate Expected Frequencies:**
   - Calculate the expected frequency for each category based on the total number of samples N and the theoretical probabilities:
     o expected_three_diff = $N \times 0.72$
     o expected_one_pair = $N \times 0.27$
     o expected_three_like = $N \times 0.01$
8. **Compute Chi-Square Statistic:**
   - Calculate the chi-square statistic using the formula: $\chi 2 = \sum (Oi - Ei)^2 / Ei$, where Oi is the observed frequency and Ei is the expected frequency for each category.
9. **Compare with Critical Value:**

- Compare the computed chi-square statistic with the critical value from the chi-square distribution table for the given significance level (e.g., $\alpha=0.05$) and degrees of freedom df = number of categories (N)- 1
- Determine if the observed distribution differs significantly from the expected distribution:
  - If $\chi 2$ < critical value, the null hypothesis (that the numbers are independent and randomly distributed) is not rejected.
  - If $\chi 2 \geq$ critical value, the null hypothesis is rejected, indicating the numbers may not be independently and randomly distributed.

PROGRAM

```
#include <stdio.h>
#include <math.h>
int main() {
   int N; // Total number of three-digit numbers
   int observed_three_diff; // Observed frequency of numbers with three different
digits
   int observed_one_pair; // Observed frequency of numbers with exactly one pair of
like digits
   int observed_three_like; // Observed frequency of numbers with three like digits
   double chi_square_critical; // Critical value for chi-square test
   // Taking input from the user
   printf("Enter the total number of three-digit numbers (N): ");
   scanf("%d", &N);
   printf("Enter the observed frequency of numbers with three different digits: ");
   scanf("%d", &observed_three_diff);
   printf("Enter the observed frequency of numbers with exactly one pair of like
digits: ");
   scanf("%d", &observed_one_pair);
   printf("Enter the observed frequency of numbers with three like digits: ");
   scanf("%d", &observed_three_like);
   printf("Enter the critical value for the chi-square test: ");
   scanf("%lf", &chi_square_critical);
   // Probabilities based on the provided image
   double p_three_diff = 0.72;
   double p_one_pair = 0.27;
   double p_three_like = 0.01;
   // Expected frequencies based on the probabilities
   double expected_three_diff = N * p_three_diff;
   double expected_one_pair = N * p_one_pair;
   double expected_three_like = N * p_three_like;
   // Compute the chi-square statistic
   double chi_square = 0;
   chi_square += pow(observed_three_diff - expected_three_diff, 2) /
expected_three_diff;
   chi_square += pow(observed_one_pair - expected_one_pair, 2) /
```

```
expected_one_pair;
    chi_square += pow(observed_three_like - expected_three_like, 2) /
expected_three_like;
    // Print the results
    printf("Chi-square statistic: %f\n", chi_square);
    if (chi_square < chi_square_critical) {
        printf("The test passed (i.e., the numbers appear to be independent).\n");
    } else {
        printf("The test failed (i.e., the numbers do not appear to be independent).\n");
    }
    return 0;
}
```

OUTPUT

```
yanil@inspiron-14:~/Desktop/Simulation_lab$ ./a.out
Enter the total number of three-digit numbers (N): 2000
Enter the observed frequency of numbers with three different digits: 1000
Enter the observed frequency of numbers with exactly one pair of like digits: 500
Enter the observed frequency of numbers with three like digits: 250
Enter the critical value for the chi-square test: 7.9
Chi-square statistic: 2782.407407
The test failed (i.e., the numbers do not appear to be independent).
```

LAB-4: IMPLEMENT LINEAR CONGRUENTIAL METHOD FOR RANDOM NUMBER GENERATION

AIM

To generate the random numbers providing the input value of constant multiplier(a), increment(c), modulus(m) & initial value seed (X0) using Linear Congruential Method.

DESCRIPTION

Linear Congruential Method:

- The linear congruential method, initially proposed by Lehmer [1951], produces a sequence of integers, X1, X2,..... between zero and m — 1 according to the following recursive relationship:
- Xi+1 = (a Xi + c) mod m, i = 0, 1, 2 ..... Equation (1)
- The initial value X0 is called the seed, a is called the constant multiplier, c is the increment, and m is the modulus.
- Case 1: If c ≠ 0 in Equation (1), the form is called the mixed congruential method.
- Case 2: When c = 0, the form is known as the multiplicative congruential method. The selection of the values for a, c, m and X0 drastically affects the statistical properties and the cycle length.
- Random numbers between zero and 1 can be generated by:
  Ri =Xi/m, i = 1, 2,.....Equation (2)

PROGRAM

```
#include <stdio.h>
#define MAX 50
int main() {
   int i, n, a, seed, c, m, r[MAX];
   printf("How many random numbers you want to generate: ");
   scanf("%d", &n);
   printf("Enter value of a: ");
   scanf("%d", &a);
   printf("Enter value of c: ");
   scanf("%d", &c);
   printf("Enter value of m: ");
   scanf("%d", &m);
   printf("Enter seed value: ");
   scanf("%d", &r[0]);
   // Check the value of c and print the corresponding message
   if (c != 0) {
      printf("c is not equal to 0 -> Mixed Congruential Method \n");
   } else {
```

```
        printf("c=0 -> Multiplicative Congruential Method \n");
    }
    for (i = 1; i <= n; i++) {
        r[i] = (a * r[i - 1] + c) % m;
    }
    printf("The random numbers are:\n");
    for (i = 1; i <= n; i++) {
        printf("%f\n", (float)r[i] / m);
    }
    return 0;
}
```

OUTPUT

```
yanil@inspiron-14:~/Desktop/Simulation_lab$ gcc 4.c
yanil@inspiron-14:~/Desktop/Simulation_lab$ ./a.out
How many random numbers you want to generate: 4
Enter value of a: 34
Enter value of c: 54
Enter value of m: 6
Enter seed value: 34
c is not equal to 0 -> Mixed Congruential Method
The random numbers are:
0.666667
0.666667
0.666667
0.666667
```

RANDOM VARIATE GENERATION

AIM

Simulating games of dice that generate discrete random variate, using random number generation.

DESCRITPION

The program uses the rand() function to generate random numbers and simulates rolling a six-sided dice multiple times, printing the result of each roll.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define DICE_SIDES 6
// Function to roll a dice
int roll_dice() {
    return (rand() % DICE_SIDES) + 1;
}
int main() {
    int num_rolls, i;
    // Seed the random number generator with the current time
    srand(time(NULL));
    // Ask the user how many times they want to roll the dice
    printf("Enter the number of times you want to roll the dice: ");
    scanf("%d", &num_rolls);
    printf("Rolling the dice %d times:\n", num_rolls);
    // Roll the dice the specified number of times
    for (i = 0; i < num_rolls; i++) {
        int result = roll_dice();
        printf("Roll %d: %d\n", i + 1, result);
    }
    return 0;
}
```

OUTPUT

```
yanil@inspiron-14:~/Desktop/Simulation_lab$ gcc 4a.c
yanil@inspiron-14:~/Desktop/Simulation_lab$ ./a.out
Enter the number of times you want to roll the dice: 5
Rolling the dice 5 times:
Roll 1: 1
Roll 2: 3
Roll 3: 5
Roll 4: 6
Roll 5: 2
```

LAB-5: SIMULATION OF SINGLE SERVER QUEUE SYSTEM USING GPSS

i. Barber Shop simulation to simulate one day of operation of a barber.

PROBLEM

Customers arrive at barber shop at the rate of 18 ± 6. Barbar serves each customer at the rate of 16 ± 4 minutes. Simulate system for queue of customer for one day (i.e. 8 hours).=480min

```
GENERATE 18,6 ; Customer arrive every 18±6 mn
QUEUE Chairs ; Enter the line
SEIZE Joe ; Capture the barber
DEPART Chairs ; Leave the line
ADVANCE 16,4 ; Get a hair cut in 16±4 mn
RELEASE Joe ; Free the barber
TERMINATE ; Leave the shop
*Simulating system for queue of customer for one day ( i.e. 8 hours).
GENERATE 480 ; Timer arrives at time = 480 mn
TERMINATE 1 ; Shut off the run; Ends the simulation run after 8 hours.
START 1 ; Start one run
```

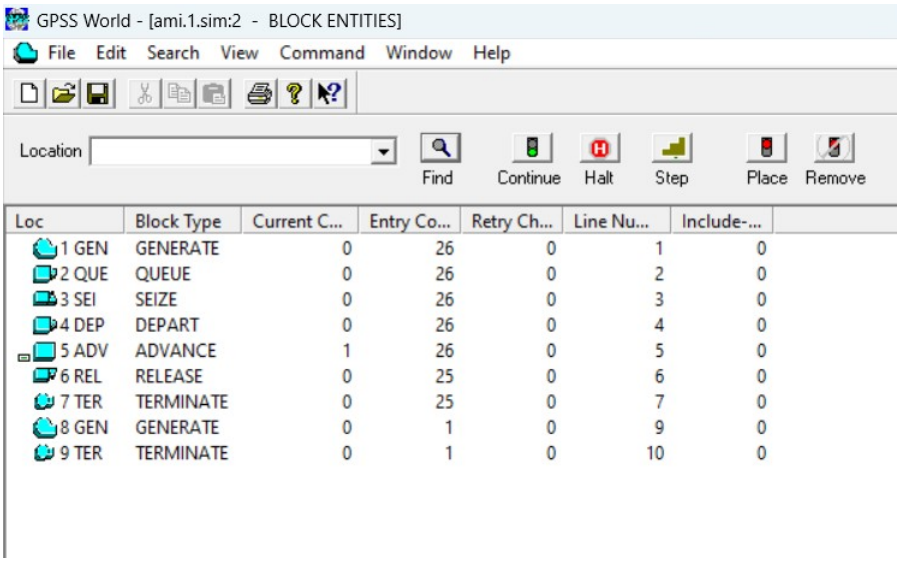
OUTPUT

GPSS World - [ami.1.1 - REPORT]

File Edit Search View Command Window Help

```
              GPSS World Simulation Report - ami.1.1


                   Monday, April 27, 2026 15:44:28

           START TIME           END TIME  BLOCKS  FACILITIES  STORAGES
                0.000            480.000     9        1          0


              NAME                        VALUE
                                       UNSPECIFIED
          CHAIRS                       10000.000
          JOE                          10001.000


 LABEL              LOC  BLOCK TYPE     ENTRY COUNT CURRENT COUNT RETRY
                    1    GENERATE            26             0       0
                    2    QUEUE               26             0       0
                    3    SEIZE               26             0       0
                    4    DEPART              26             0       0
                    5    ADVANCE             26             1       0
                    6    RELEASE             25             0       0
                    7    TERMINATE           25             0       0
                    8    GENERATE             1             0       0
                    9    TERMINATE            1             0       0


FACILITY         ENTRIES  UTIL.   AVE. TIME AVAIL. OWNER PEND INTER RETRY DELAY
 JOE                 26    0.863      15.926  1       27    0    0     0      0


QUEUE              MAX CONT. ENTRY ENTRY(0) AVE.CONT. AVE.TIME   AVE.(-0) RETRY
 CHAIRS              1    0     26     15     0.078      1.441      3.407   0


FEC XN   PRI         BDT      ASSEM  CURRENT  NEXT  PARAMETER    VALUE
    27    0         482.664     27      5      6
    28    0         482.707     28      0      1
    29    0         960.000     29      0      8
```

BLOCK ENTITIES



Loc	Block Type	Current C...	Entry Co...	Retry Ch...	Line Nu...	Include-...
1 GEN	GENERATE	0	26	0	1	0
2 QUE	QUEUE	0	26	0	2	0
3 SEI	SEIZE	0	26	0	3	0
4 DEP	DEPART	0	26	0	4	0
5 ADV	ADVANCE	1	26	0	5	0
6 REL	RELEASE	0	25	0	6	0
7 TER	TERMINATE	0	25	0	7	0
8 GEN	GENERATE	0	1	0	9	0
9 TER	TERMINATE	0	1	0	10	0

ii. Barber Shop simulation (simple).

PROBLEM

Customers arrive at barber shop at the rate of 300 ± 100. Mechanic serves each customer at the rate of 400 ± 200 minutes. Simulate system for queue of customer and measure of waiting time for 1000 customers.

MODEL

GENERATE 300,100 ; Create next customer
QUEUE Barber ; Begin queue time
SEIZE Barber ; Own or wait for barber
DEPART Barber ; End queue time
ADVANCE 400,200 ; Haircut takes a few minutes
RELEASE Barber ; Haircut done. Give up the barber.
TERMINATE 1 ; Customer leaves
START 1000; for 1000 customers OR it can be provided from user instead of writing this line of code

OUTPUT

GPSS World - [simam.2.1 - REPORT]

File Edit Search View Command Window Help

```
GPSS World Simulation Report - simam.2.1

Monday, April 27, 2026 17:02:42

START TIME          END TIME  BLOCKS  FACILITIES  STORAGES
    0.000        4793617.629     7        1          0

NAME                        VALUE
BARBER                   10000.000

LABEL              LOC  BLOCK TYPE     ENTRY COUNT CURRENT COUNT RETRY
                   1    GENERATE         24014           0       0
                   2    QUEUE            24014       16013       0
                   3    SEIZE             8001           1       0
                   4    DEPART            8000           0       0
                   5    ADVANCE           8000           0       0
                   6    RELEASE           8000           0       0
                   7    TERMINATE         8000           0       0

FACILITY         ENTRIES  UTIL.   AVE. TIME AVAIL. OWNER PEND INTER RETRY DELAY
 BARBER            8001    1.000     599.094  1     8001    0     0     0   16013

QUEUE              MAX CONT. ENTRY ENTRY(0) AVE.CONT. AVE.TIME   AVE.(-0) RETRY
 BARBER           16014 16014  24014      1  8009.812 1598899.647 1598966.232   0

CEC XN   PRI          M1       ASSEM  CURRENT  NEXT  PARAMETER    VALUE
  8001    0     1597468.342    8001      3      4

FEC XN   PRI         BDT       ASSEM  CURRENT  NEXT  PARAMETER    VALUE
 24015    0     4793794.589    24015     0      1
```

BLOCK ENTITIES

Loc	Block Type	Current C...	Entry Co...	Retry Ch...	Line Nu...	Include-...
1 GEN	GENERATE	0	1347	0	1	0
2 QUE	QUEUE	346	1347	0	2	0
3 SEI	SEIZE	1	1001	0	3	0
4 DEP	DEPART	0	1000	0	4	0
5 ADV	ADVANCE	0	1000	0	5	0
6 REL	RELEASE	0	1000	0	6	0
7 TER	TERMINATE	0	1000	0	7	0

iii. Mechanic Shop simulation

PROBLEM

Customers arrive at mechanic shop at the rate of 300 ± 200. Mechanic serves each customer at the rate of 200 ± 50 minutes. Simulate system for queue of customer and measure of waiting time for 1000 customers.

MODEL

```
GENERATE 300 200 ; Create next customer
QUEUE MECHANIC
SEIZE MECHANIC ; Own mechanic(resource)
DEPART MECHANIC
```

```
ADVANCE 200 50 ; Mechanic Takes some Time
RELEASE MECHANIC ; Release the resource
TERMINATE 1 ; End simulation
START 1000
```

OUTPUT

GPSS World - [Untitled Model 3.1.1 - REPORT]

File Edit Search View Command Window Help

```
              GPSS World Simulation Report - Untitled Model 3.1.1

                   Monday, April 27, 2026 17:07:12

           START TIME          END TIME  BLOCKS  FACILITIES  STORAGES
               0.000          99966.602     7        1          0

              NAME                        VALUE
           MECHANIC                    10000.000

 LABEL              LOC  BLOCK TYPE     ENTRY COUNT CURRENT COUNT RETRY
                    1    GENERATE           501            0       0
                    2    QUEUE              501            0       0
                    3    SEIZE              501            1       0
                    4    DEPART             500            0       0
                    5    ADVANCE            500            0       0
                    6    RELEASE            500            0       0
                    7    TERMINATE          500            0       0

FACILITY          ENTRIES  UTIL.   AVE. TIME AVAIL. OWNER PEND INTER RETRY DELAY
 MECHANIC            501    0.749     149.487  1      501    0    0     0      0

QUEUE               MAX CONT. ENTRY ENTRY(0) AVE.CONT. AVE.TIME   AVE.(-0) RETRY
 MECHANIC             1    1    501    360     0.047      9.451     33.582   0

CEC XN    PRI          M1       ASSEM  CURRENT  NEXT  PARAMETER    VALUE
   501     0        99965.600    501      3      4

FEC XN    PRI          BDT      ASSEM  CURRENT  NEXT  PARAMETER    VALUE
   502     0       100173.766    502      0      1
```

BLOCK ENTITIES

GPSS World - [Untitled Model 3.1.sim:2 - BLOCK ENTITIES]

File Edit Search View Command Window Help

Location | Find | Continue | Halt | Step | Place | Remove

Loc	Block Type	Current C...	Entry Co...	Retry Ch...	Line Nu...	Include-...
1 GEN	GENERATE	0	501	0	1	0
2 QUE	QUEUE	0	501	0	2	0
3 SEI	SEIZE	1	501	0	3	0
4 DEP	DEPART	0	500	0	4	0
5 ADV	ADVANCE	0	500	0	5	0
6 REL	RELEASE	0	500	0	6	0
7 TER	TERMINATE	0	500	0	7	0

iv. Telephone System Simulation

PROBLEM

Construct a GPSS simulation model to represent a basic telephone system with the following characteristics: Calls are generated randomly, arriving on average every 100 time units with a standard deviation of 20. The system has one phone line to process calls. If the line is available, calls are handled immediately; otherwise, they join a queue. Each call takes an average of 50 time units to process, with a standard deviation of 10. Once a call is completed, the line becomes available for the next queued call. The simulation concludes after each call is processed.

MODEL

```
GENERATE 100,20 ; Generate calls every 100 units with a 20 unit standard deviation
QUEUE Callqueue ; Queue for incoming calls
SEIZE Phone ; Attempt to seize a phone line
DEPART Callqueue; Depart from the queue
ADVANCE 50,10 ; Call duration with a mean of 50 units and a 10 unit standard deviation
RELEASE Phone; Release the phone line
TERMINATE 1 ; Terminate the entity (call)
```

OUTPUT

GPSS World - [telephone.8.1 - REPORT]

File Edit Search View Command Window Help

```
              GPSS World Simulation Report - telephone.8.1


                   Monday, April 27, 2026 17:24:14

           START TIME            END TIME  BLOCKS  FACILITIES  STORAGES
                0.000             171.314     7        1          0


              NAME                        VALUE
           LINEQUEUE                    10000.000
           PHONE                        10001.000


 LABEL                LOC  BLOCK TYPE     ENTRY COUNT CURRENT COUNT RETRY
                      1    GENERATE             1             0       0
                      2    QUEUE                1             0       0
                      3    SEIZE                1             0       0
                      4    DEPART               1             0       0
                      5    ADVANCE              1             0       0
                      6    RELEASE              1             0       0
                      7    TERMINATE            1             0       0


FACILITY          ENTRIES  UTIL.   AVE. TIME AVAIL. OWNER PEND INTER RETRY DELAY
 PHONE                1    0.336      57.620  1        0    0    0     0      0


QUEUE               MAX CONT. ENTRY ENTRY(0) AVE.CONT. AVE.TIME   AVE.(-0) RETRY
 LINEQUEUE            1    0      1      1     0.000      0.000      0.000   0


FEC XN    PRI          BDT      ASSEM  CURRENT  NEXT  PARAMETER    VALUE
     2     0         224.803      2      0      1
```

BLOCK ENTITIES

GPSS World - [telephone.9.sim:2 - BLOCK ENTITIES]

File Edit Search View Command Window Help

Location | Find | Continue | Halt | Step | Place | Remove

Loc	Block Type	Current C...	Entry Co...	Retry Ch...	Line Nu...	Include-...
1 GEN	GENERATE	0	1	0	1	0
2 QUE	QUEUE	0	1	0	2	0
3 SEI	SEIZE	0	1	0	3	0
4 DEP	DEPART	0	1	0	4	0
5 ADV	ADVANCE	0	1	0	5	0
6 REL	RELEASE	0	1	0	6	0
7 TER	TERMINATE	0	1	0	7	0

v. Turnstile of Football Stadium.

PROBLEM

Spectators arrive at a turnstile of a football stadium every 7±7 seconds and queue for admittance. The time to pass through is evenly distributed at 5±3 seconds. A model is required to determine the time taken by 300 people to pass through the turnstile.

MODEL

```
GENERATE 7,7 ; People arrive
QUEUE Turn ; Enter queue
SEIZE Turn ; Acquire turnstile
DEPART Turn ; Depart the queue
ADVANCE 5,3 ; Use tulrnstile
RELEASE Turn ; Leave turnstile
TERMINATE 1 ; One Spectator Enters
```

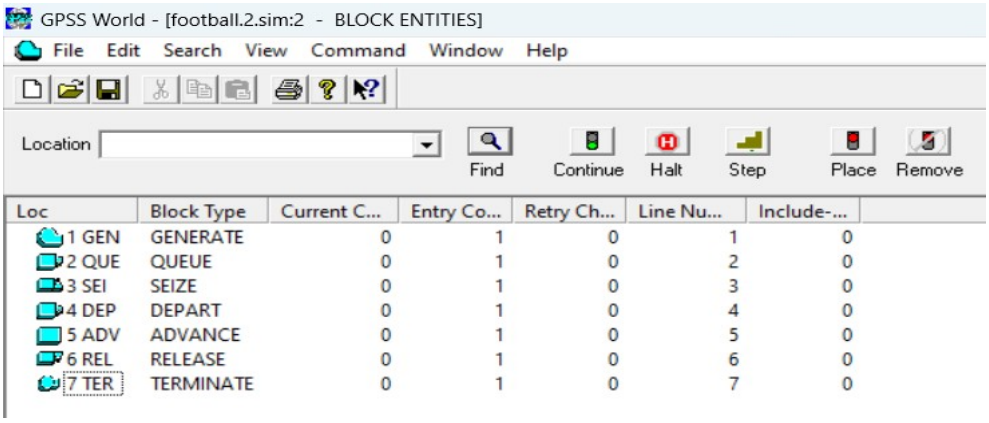
OUTPUT

GPSS World - [football.2.1 - REPORT]

File Edit Search View Command Window Help

```
              GPSS World Simulation Report - football.2.1


                   Monday, April 27, 2026 17:30:30

           START TIME           END TIME  BLOCKS  FACILITIES  STORAGES
                0.000             25.895     7        1          0


              NAME                        VALUE
          TURNQUEUE                    10000.000
          TURNSTILE                    10001.000


 LABEL              LOC  BLOCK TYPE     ENTRY COUNT CURRENT COUNT RETRY
                    1    GENERATE             1             0       0
                    2    QUEUE                1             0       0
                    3    SEIZE                1             0       0
                    4    DEPART               1             0       0
                    5    ADVANCE              1             0       0
                    6    RELEASE              1             0       0
                    7    TERMINATE            1             0       0


FACILITY         ENTRIES  UTIL.   AVE. TIME AVAIL. OWNER PEND INTER RETRY DELAY
 TURNSTILE            1    0.349       9.048  1        0    0    0     0      0


QUEUE              MAX CONT. ENTRY ENTRY(0) AVE.CONT. AVE.TIME   AVE.(-0) RETRY
 TURNQUEUE           1    0      1      1     0.000      0.000      0.000   0


FEC XN   PRI         BDT      ASSEM  CURRENT  NEXT  PARAMETER    VALUE
     2    0          32.402      2      0      1
```

BLOCK ENTITIES



Loc	Block Type	Current C...	Entry Co...	Retry Ch...	Line Nu...	Include-...
1 GEN	GENERATE	0	1	0	1	0
2 QUE	QUEUE	0	1	0	2	0
3 SEI	SEIZE	0	1	0	3	0
4 DEP	DEPART	0	1	0	4	0
5 ADV	ADVANCE	0	1	0	5	0
6 REL	RELEASE	0	1	0	6	0
7 TER	TERMINATE	0	1	0	7	0

vi. Manufacturing Shop.

PROBLEM

A machine tool in a manufacturing shop is turning out parts at the rate of every 5 minutes. As they are finished, the parts are turned over to an inspector who takes 4±3 minutes to examine each one and rejects about 10% of the parts as faulty. Each part will be represented by a transaction and the base time unit for the system is chosen as 1 minute. Simulate for 100 parts to leave the system.

MODEL 1

```
SIMULATE
GENERATE 5,0
ADVANCE 4,3
TRANSFER 0.1,ACC,REJ
ACC TERMINATE 1
REJ TERMINATE 1
START 100
```

OUTPUT

GPSS World - [Manufacture.3.1 - REPORT]

File Edit Search View Command Window Help

```
              GPSS World Simulation Report - Manufacture.3.1


                   Monday, April 27, 2026 17:33:34

           START TIME           END TIME  BLOCKS  FACILITIES  STORAGES
                0.000           1605.450     5        0          0


              NAME                        VALUE
           ACC                             4.000
           REJ                             5.000


 LABEL              LOC  BLOCK TYPE     ENTRY COUNT CURRENT COUNT RETRY
                    1    GENERATE           200            0       0
                    2    ADVANCE            200            0       0
                    3    TRANSFER           200            0       0
ACC                 4    TERMINATE          181            0       0
REJ                 5    TERMINATE           19            0       0


FEC XN   PRI         BDT      ASSEM  CURRENT  NEXT  PARAMETER    VALUE
   201    0        1608.000    201      0      1
```

BLOCK ENTITIES

GPSS World - [Manufacture.4.sim:2 - BLOCK ENTITIES]

File Edit Search View Command Window Help

Location | Find | Continue | Halt | Step | Place | Remove

Loc	Block Type	Current C...	Entry Co...	Retry Ch...	Line Nu...	Include-...
1 GEN	GENERATE	0	200	0	2	0
2 ADV	ADVANCE	0	200	0	3	0
3 TRA	TRANSFER	0	200	0	4	0
ACC	TERMINATE	0	181	0	5	0
REJ	TERMINATE	0	19	0	6	0

MODEL 2

Modify the previous examples using facilities: (SEIZE &RELEASE)
There is only one inspector

```
SIMULATE
GENERATE 5,0
SEIZE Insp
ADVANCE 4,3
RELEASE Insp
TRANSFER 0.1,Acc,Rej
Acc TERMINATE 1
Rej TERMINATE 1
START 1000
```

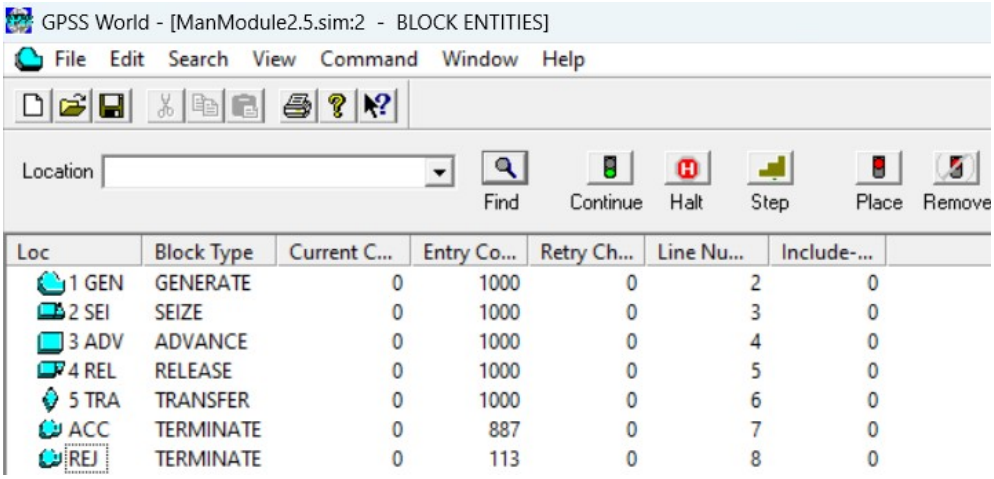
OUTPUT

GPSS World - [ManModule2.5.1 - REPORT]

File Edit Search View Command Window Help

```
              GPSS World Simulation Report - ManModule2.5.1


                  Monday, April 27, 2026 17:40:37

           START TIME          END TIME  BLOCKS  FACILITIES  STORAGES
                0.000          5001.738     7        1          0


              NAME                        VALUE
          ACC                              6.000
          INSP                         10000.000
          REJ                              7.000


 LABEL              LOC  BLOCK TYPE     ENTRY COUNT CURRENT COUNT RETRY
                    1    GENERATE          1000             0       0
                    2    SEIZE             1000             0       0
                    3    ADVANCE           1000             0       0
                    4    RELEASE           1000             0       0
                    5    TRANSFER          1000             0       0
ACC                 6    TERMINATE          887             0       0
REJ                 7    TERMINATE          113             0       0


FACILITY         ENTRIES  UTIL.   AVE. TIME AVAIL. OWNER PEND INTER RETRY DELAY
 INSP              1000    0.803       4.015  1        0    0    0     0      0


FEC XN   PRI         BDT      ASSEM  CURRENT  NEXT  PARAMETER    VALUE
  1001    0        5005.000   1001      0      1
```

BLOCK ENTITIES



Loc	Block Type	Current C...	Entry Co...	Retry Ch...	Line Nu...	Include-...
1 GEN	GENERATE	0	1000	0	2	0
2 SEI	SEIZE	0	1000	0	3	0
3 ADV	ADVANCE	0	1000	0	4	0
4 REL	RELEASE	0	1000	0	5	0
5 TRA	TRANSFER	0	1000	0	6	0
ACC	TERMINATE	0	887	0	7	0
REJ	TERMINATE	0	113	0	8	0

MODEL 3

Modify the previous example using storages. (ENTER & LEAVE)
There are 3 inspectors

```
GENERATE 5,0
ENTER Ins,1
ADVANCE 4,3
LEAVE Ins,1
TRANSFER 0.1,Acc,Rej
Acc TERMINATE 1
Rej TERMINATE 1
Ins STORAGE 3
START 1000
```

OUTPUT

GPSS World - [ManModule3.9.1 - REPORT]

File Edit Search View Command Window Help

Help

```
              GPSS World Simulation Report - ManModule3.9.1


                  Monday, April 27, 2026 17:50:46

           START TIME          END TIME  BLOCKS  FACILITIES  STORAGES
                0.000          5001.837     7        0          1


              NAME                        VALUE
          ACC                              6.000
          INS                          10000.000
          REJ                              7.000


 LABEL              LOC  BLOCK TYPE     ENTRY COUNT CURRENT COUNT RETRY
                    1    GENERATE          1000             0       0
                    2    ENTER             1000             0       0
                    3    ADVANCE           1000             0       0
                    4    LEAVE             1000             0       0
                    5    TRANSFER          1000             0       0
ACC                 6    TERMINATE           96             0       0
REJ                 7    TERMINATE          904             0       0


STORAGE            CAP. REM. MIN. MAX.  ENTRIES AVL.  AVE.C. UTIL. RETRY DELAY
 INS                 3    3   0     2     1000   1    0.790  0.263    0    0


FEC XN   PRI         BDT      ASSEM  CURRENT  NEXT  PARAMETER    VALUE
  1001    0        5005.000   1001      0      1
```

BLOCK ENTITIES

GPSS World - [ManModule3.9.sim:2 - BLOCK ENTITIES]

File Edit Search View Command Window Help

Location Find Continue Halt Step Place Remove

Loc	Block Type	Current C...	Entry Co...	Retry Ch...	Line Nu...	Include-...
1 GEN	GENERATE	0	1000	0	1	0
2 ENT	ENTER	0	1000	0	2	0
3 ADV	ADVANCE	0	1000	0	3	0
4 LEA	LEAVE	0	1000	0	4	0
5 TRA	TRANSFER	0	1000	0	5	0
ACC	TERMINATE	0	96	0	6	0
REJ	TERMINATE	0	904	0	7	0

MODEL 4

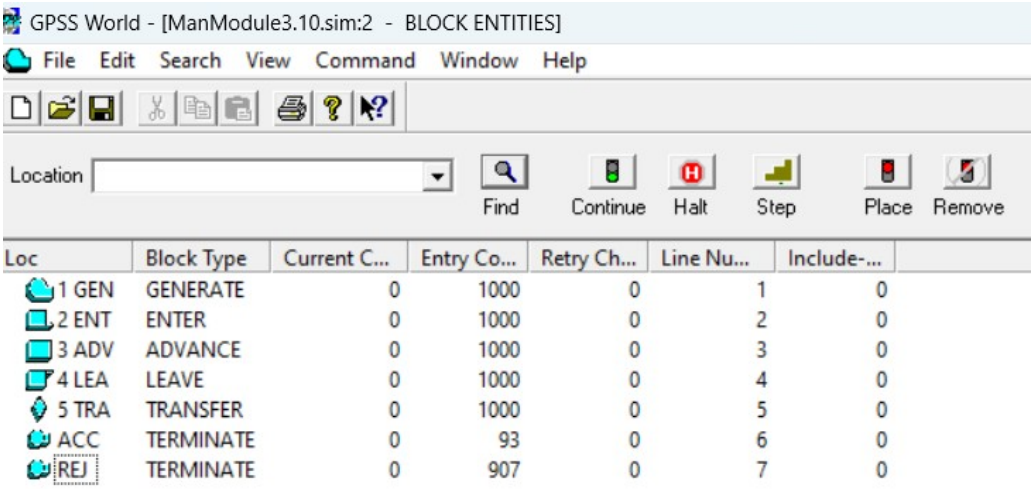
Modifying the previous example using storage & keeping queue statistics: There are 3 inspectors.

GPSS World - [ManModule3.10.1 - REPORT]

File Edit Search View Command Window Help

```
              GPSS World Simulation Report - ManModule3.10.1


                   Monday, April 27, 2026 17:55:27

           START TIME            END TIME  BLOCKS  FACILITIES  STORAGES
                0.000            6005.321     7        0          1


              NAME                          VALUE
           ACC                               6.000
           INS                           10000.000
           REJ                               7.000


 LABEL                LOC  BLOCK TYPE     ENTRY COUNT CURRENT COUNT RETRY
                      1    GENERATE          1000              0       0
                      2    ENTER             1000              0       0
                      3    ADVANCE           1000              0       0
                      4    LEAVE             1000              0       0
                      5    TRANSFER          1000              0       0
ACC                   6    TERMINATE           93              0       0
REJ                   7    TERMINATE          907              0       0


STORAGE              CAP. REM. MIN. MAX.  ENTRIES AVL.  AVE.C. UTIL. RETRY DELAY
 INS                   4    4   0     2     1000   1    0.656  0.164    0    0


FEC XN   PRI          BDT      ASSEM  CURRENT  NEXT  PARAMETER    VALUE
  1001    0         6006.000   1001      0      1
```

BLOCK ENTITIES



Loc	Block Type	Current C...	Entry Co...	Retry Ch...	Line Nu...	Include-...
1 GEN	GENERATE	0	1000	0	1	0
2 ENT	ENTER	0	1000	0	2	0
3 ADV	ADVANCE	0	1000	0	3	0
4 LEA	LEAVE	0	1000	0	4	0
5 TRA	TRANSFER	0	1000	0	5	0
ACC	TERMINATE	0	93	0	6	0
REJ	TERMINATE	0	907	0	7	0

MODEL 5

Simulate system for T time units and not for k Xacts to finish a service,

- This line states the objective of the simulation. It aims to run for a specific duration (T time units) rather than until a certain number of transactions (Xacts) complete a service.
  For example, simulate for 8 hours (base time unit is 1 minute)
- This provides an example of the simulation duration: 8 hours, where each time unit represents 1 minute.

```
GENERATE 5
QUEUE Insq
ENTER Ins,1
DEPART Insq
ADVANCE 4,3
LEAVE Ins,1
TRANSFER 0.1,Acc,Rej
Acc TERMINATE 0
Rej TERMINATE 0
GENERATE 480
TERMINATE 1
Ins STORAGE 3
START 1
```

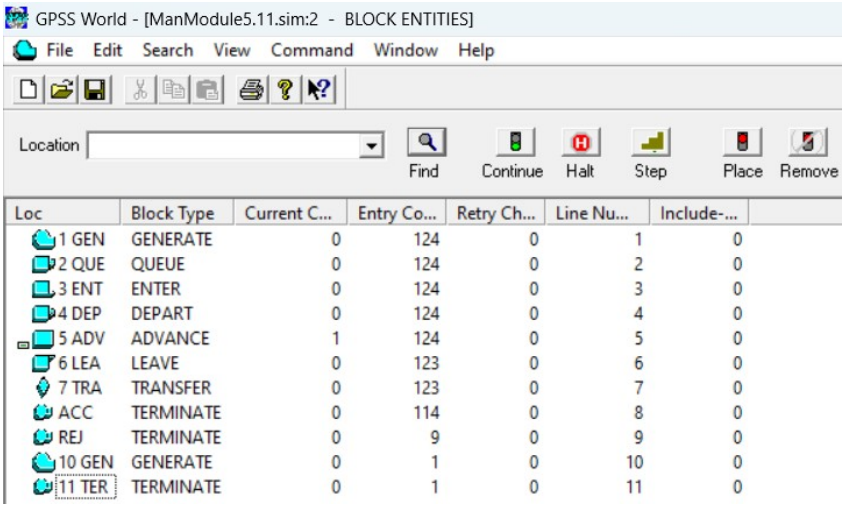
OUPTUT

GPSS World - [ManModule5.11.1 - REPORT]

File Edit Search View Command Window Help

```
              GPSS World Simulation Report - ManModule5.11.1


                   Monday, April 27, 2026 17:58:43

           START TIME           END TIME  BLOCKS  FACILITIES  STORAGES
                0.000            500.000    11        0          1


              NAME                        VALUE
          ACC                              8.000
          INS                          10000.000
          INSQ                         10001.000
          REJ                              9.000


 LABEL              LOC  BLOCK TYPE     ENTRY COUNT CURRENT COUNT RETRY
                    1    GENERATE           124             0       0
                    2    QUEUE              124             0       0
                    3    ENTER              124             0       0
                    4    DEPART             124             0       0
                    5    ADVANCE            124             1       0
                    6    LEAVE              123             0       0
                    7    TRANSFER           123             0       0
ACC                 8    TERMINATE          114             0       0
REJ                 9    TERMINATE            9             0       0
                   10    GENERATE             1             0       0
                   11    TERMINATE            1             0       0


QUEUE              MAX CONT. ENTRY ENTRY(0) AVE.CONT. AVE.TIME   AVE.(-0) RETRY
 INSQ                1    0    124    124     0.000      0.000      0.000   0


STORAGE            CAP. REM. MIN. MAX.  ENTRIES AVL.  AVE.C. UTIL. RETRY DELAY
 INS                 3    2    0     2      124   1    1.230  0.410    0     0


CEC XN   PRI          M1      ASSEM  CURRENT  NEXT  PARAMETER    VALUE
   126    0         500.000    126      0      1


FEC XN   PRI         BDT      ASSEM  CURRENT  NEXT  PARAMETER    VALUE
   125    0         502.835    125      5      6
   127    0        1000.000    127      0     10
```

BLOCK ENTITIES



Loc	Block Type	Current C...	Entry Co...	Retry Ch...	Line Nu...	Include-...
1 GEN	GENERATE	0	124	0	1	0
2 QUE	QUEUE	0	124	0	2	0
3 ENT	ENTER	0	124	0	3	0
4 DEP	DEPART	0	124	0	4	0
5 ADV	ADVANCE	1	124	0	5	0
6 LEA	LEAVE	0	123	0	6	0
7 TRA	TRANSFER	0	123	0	7	0
ACC	TERMINATE	0	114	0	8	0
REJ	TERMINATE	0	9	0	9	0
10 GEN	GENERATE	0	1	0	10	0
11 TER	TERMINATE	0	1	0	11	0